

Module 3
SCTP DSAI DS2F
Coaching

Moneyball
Analytics



Centre for Professional and
Continuing Education



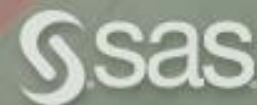
SkillsFuture Career Transition Programme (SCTP)

Advanced Professional Certificate in
Data Science and AI

Get the Skills, Tools and Mindset to
Kickstart Your Tech Career

Moneyball Analytics

Even a tiny insight can lead to a huge advantage on the field



Moneyball Analytics

Even a tiny insight can lead to a huge advantage on the field

AutoML

URL: <https://www.automl.org/automl/>



Moneyball Analytics

Even a tiny insight can lead to a huge advantage on the field

AutoML

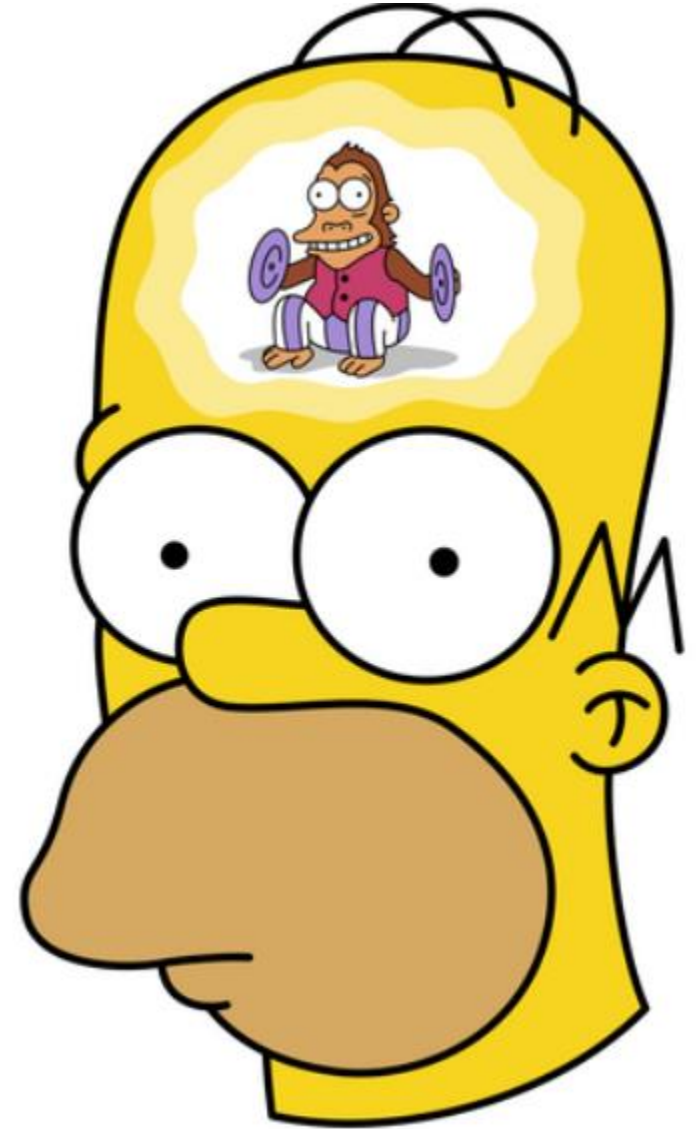
URL: <https://www.automl.org/automl/>

Models I used:



Moneyball Analytics

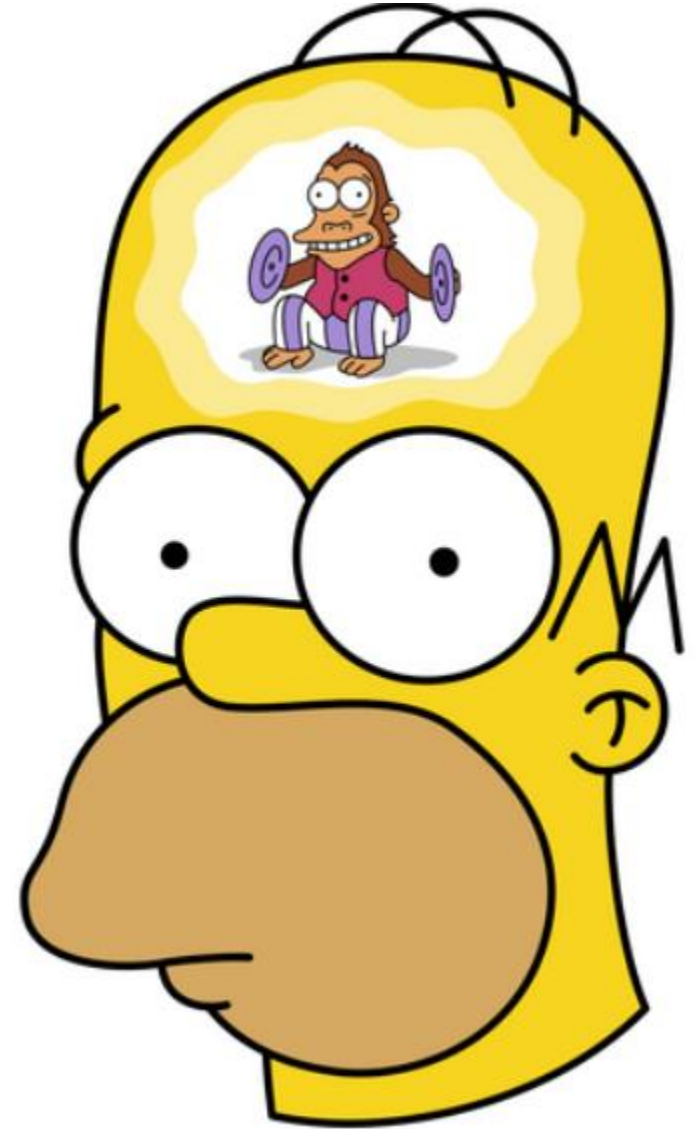
My Challenges & Solutions



Moneyball Analytics

My Challenges & Solutions

Challenge – Variety of Machine Learning models

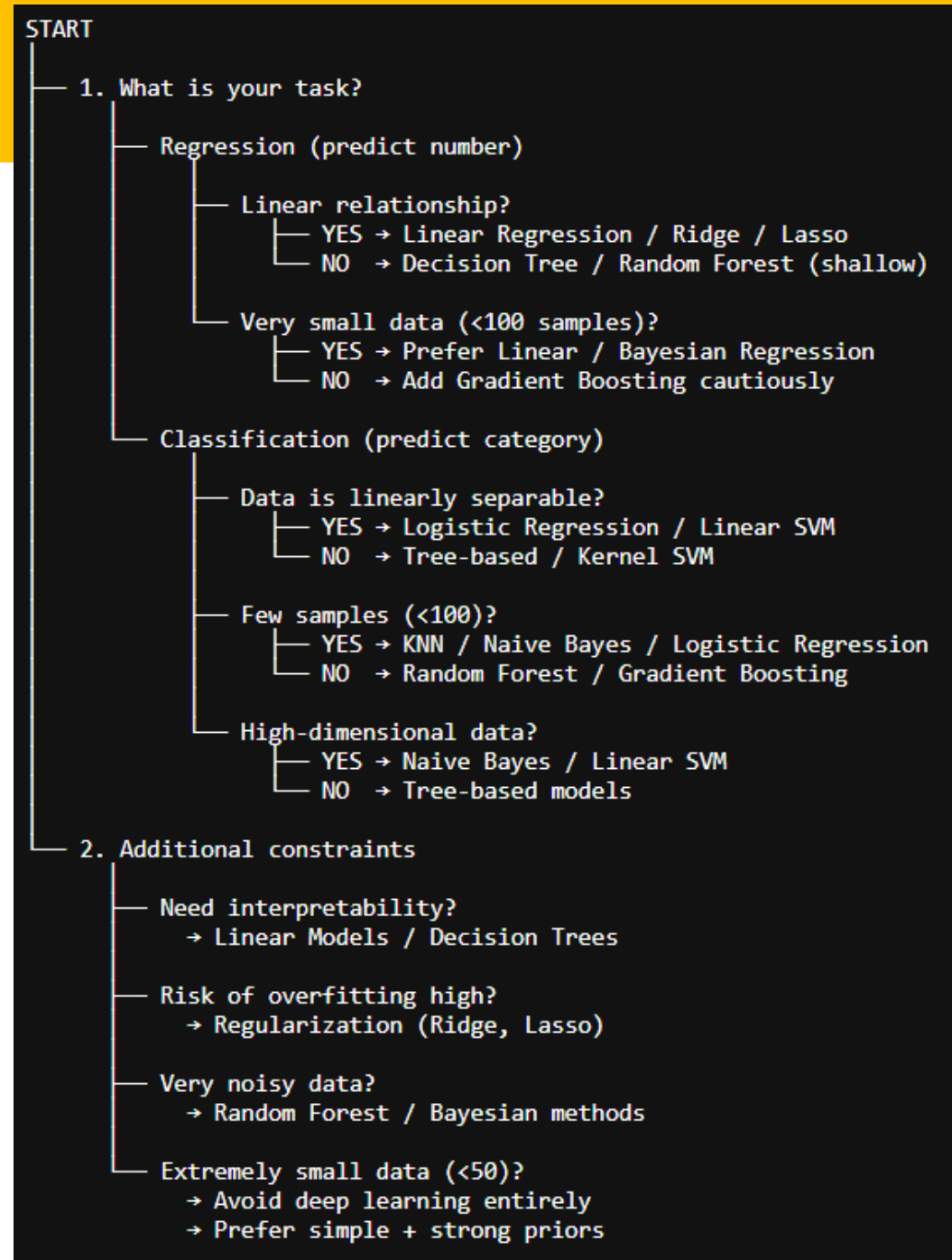


Moneyball Analytics

My Challenges & Solutions

Challenge – Variety of Machine Learning models

Solution – Use diagrams



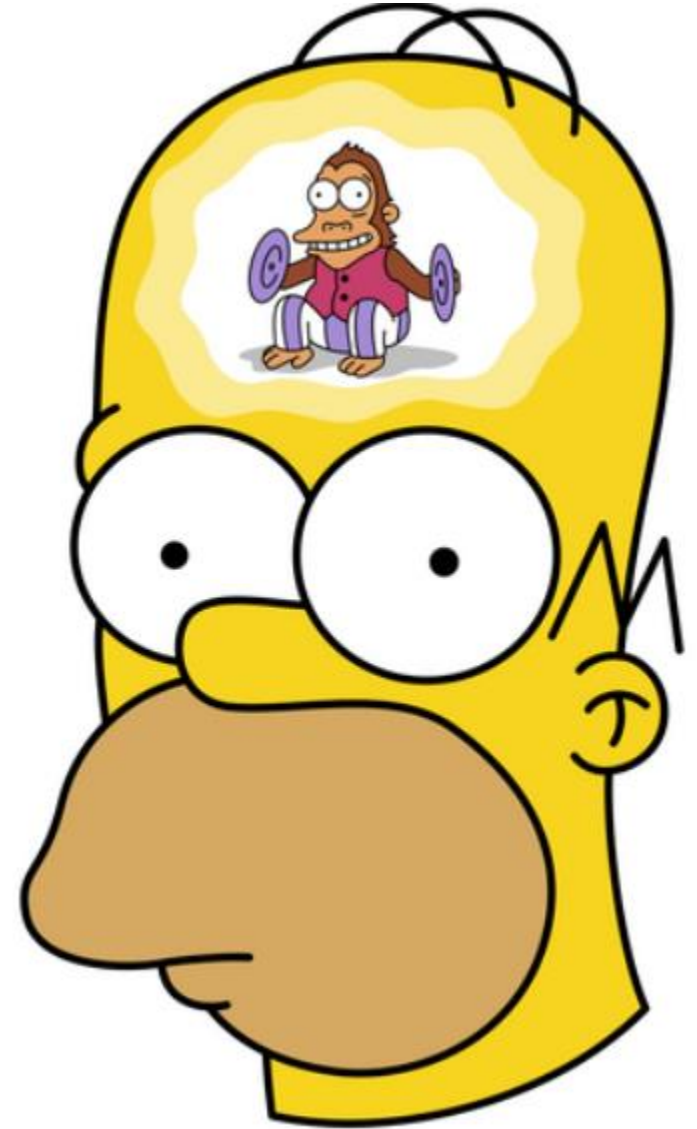
Moneyball Analytics

My Challenges & Solutions

Challenge – Variety of Machine Learning models

Solution – Use diagrams

Challenge – Lost in math and algorithm



Moneyball Analytics

My Challenges & Solutions

Challenge – Variety of Machine Learning Algorithms
Solution – Use diagrams

Challenge – Lost in math and statistics
Solution – Use AI to simplify

9.7 Comparative training table

Algorithm family	Typical data	Strengths	Weaknesses	Good default use
Linear / logistic	structured, sparse text	interpretable, fast, strong baseline	misses complex nonlinearity	first supervised baseline
Trees	structured	intuitive rules	overfits easily	quick prototype
Random forest	structured	robust, low-preprocessing baseline	heavier inference	nonlinear tabular baseline
Boosted trees	structured	top tabular performance	more tuning	serious tabular modeling
SVM	structured, sparse high-dimensional	strong margins	scale/tuning cost	niche small/medium problems
k-NN	structured/embedded	simple local baseline	poor scalability	sanity-check baseline
Neural nets	unstructured, large data	expressive, transfer learning	compute and tuning	text/image/audio tasks
k-means	unlabeled tabular	fast segmentation	assumes cluster geometry	exploratory clustering
DBSCAN	unlabeled tabular/spatial	finds noise and arbitrary shapes	sensitive parameters	anomaly-like segmentation
PCA/SVD	high-dimensional	compression	interpretability	preprocessing and visualization
RL methods	sequential decision problems	learns policies with delayed reward	high variance, evaluation difficulty	only when actions change future outcomes

Moneyball Analytics

My Challenges & Solutions

Challenge – Variety of Machine Learning models

Solution – Use diagrams

Challenge – Lost in math and algorithm

Solution – Use AI to simplify

```
# =====  
# MODEL 2 CONFIGURATION – ADJUST THESE HYPERPARAMETERS  
# =====  
  
MODEL2_N_TRIALS = 50  
MODEL2_CV_SPLITS = 10  
MODEL2_USE_GROUP_CV = False  
MODEL2_USE_GROUP_KFOLD = True  
MODEL2_GROUP_COLUMN = "yearID"  
MODEL2_RANDOM_SEED = 42  
MODEL2_OUTPUT_FILE = "prediction_submission_2.csv"  
  
# Allowed algorithm families in the random search pool.  
MODEL2_CANDIDATE_POOL = [  
    "ridge",  
    "lasso",  
    "elasticnet",  
    "random_forest",  
    "extra_trees",  
    "gradient_boosting",  
    "hist_gradient_boosting",  
    "lightgbm",  
]
```

Moneyball Analytics

My Challenges & Solutions

Challenge – Variety of Machine Learning models

Solution – Use diagrams

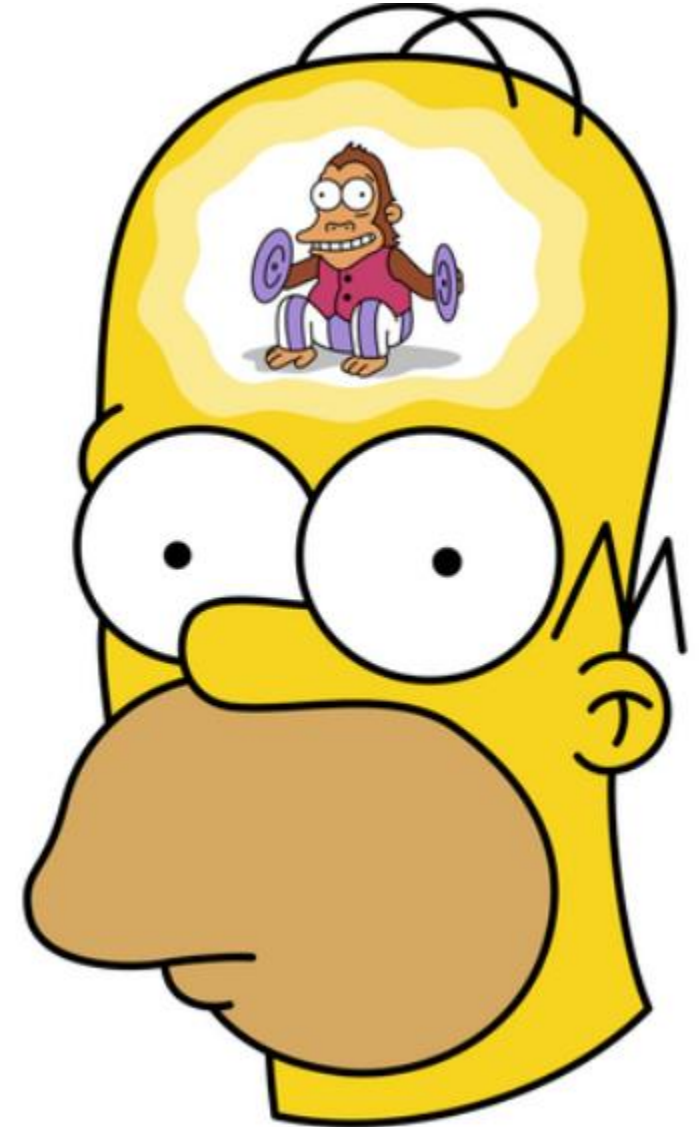
Challenge – Lost in math and algorithm

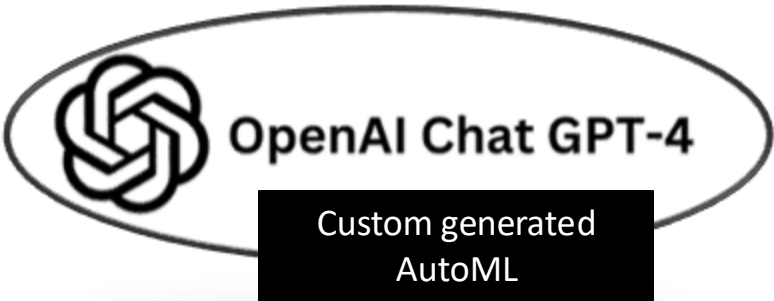
Solution – Use AI to simplify

Challenge – Domain knowledge (MLB metrics)

Solution – NotebookLM

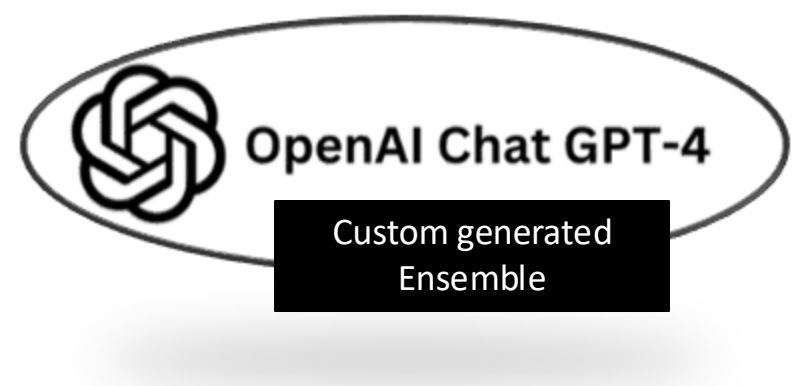
(URL: <https://notebooklm.google.com/notebook/d7ccb014-dbc0-488a-ab59-4fba63ef5863>)



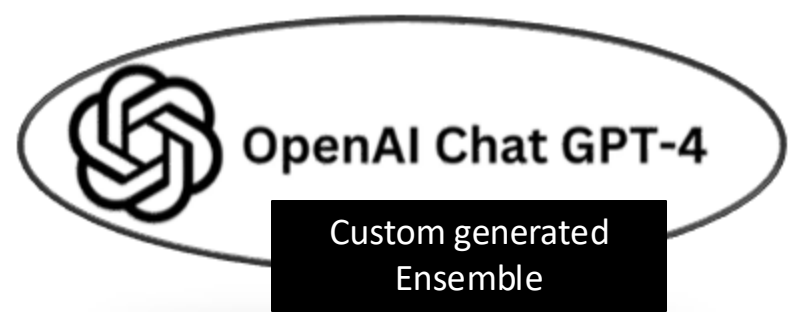


Hyper-Param \ # of runs	1	2	3	4	5
TIME_LEFT_FOR_THIS_TASK	1800	2700	3600	5400	7200
PER_RUN_TIME_LIMIT	120	120	120	120	180
MEMORY_LIMIT_MB	4096	4096	4096	4096	4096
ENSEMBLE_SIZE	25	25	50	50	25
RESAMPLING_FOLDS	5	5	5	5	5
N_JOBS	4	4	4	4	4
PRIVATE SCORE (TRUNC)	3.30	3.13	3.86	3.16	3.14

Hyper-Param \ # of runs	1	2	3	4	5
N_TRIALS	18	25	30	50	80
CV_SPLITS	5	5	10	10	8
USE_GROUP_CV	True	True	True	False	True
USE_GROUP_KFOLD			True	True	False
GROUP_COLUMN		yearID	yearID	yearID	
RANDOM_SEED	42	42	42	42	42
BEST ALGORITHM	Lasso	Lasso	Lasso	Lasso	Lasso
PRIVATE SCORE (TRUNC)	3.16	3.16	3.16	3.29	3.36



Hyper-Param \ # of runs	1	2	3	4	5
TOTAL_TRIALS	24	30	35	50	80
TOP_POOL_SIZE	8	10	5	10	15
SELECT_TOP_K	5	5	5		
CV_SPLITS	5	5	5	10	8
USE_GROUP_CV	True	True	True	True	True
ENSEMBLE_SIZE	5	4	3	3	5
GROUP_COLUMN	yearID	yearID	yearID		
RANDOM_STATE	42	42	42		
RANDOM_SEED	123	123	123	123	123
WEIGHTING_METHOD	Inverse MAE				
ENSEMBLE SELECTION	Ridge 4 ElasticNet	Ridge 3 ElasticNet	Ridge 3	Ridge 3	Ridge 4 ElasticNet
PRIVATE SCORE (TRUNC)	3.22	3.21	3.20	3.24	3.22



Best Personal Private Score

- 3.13 (Auto-SKLearn)

Top Team 5 Private Scores

- 2.99176 (Ridge) by Lewis
- 3.00000 (Ridge) by Lewis

Hyper-Param \ # of runs	1	2	3	4	5
TOTAL_TRIALS	24	30	35	50	80
TOP_POOL_SIZE	8	10	5	10	15
SELECT_TOP_K	5	5	5		
CV_SPLITS	5	5	5	10	8
USE_GROUP_CV	True	True	True	True	True
ENSEMBLE_SIZE	5	4	3	3	5
GROUP_COLUMN	yearID	yearID	yearID		
RANDOM_STATE	42	42	42		
RANDOM_SEED	123	123	123	123	123
WEIGHTING_METHOD	Inverse MAE				
ENSEMBLE SELECTION	Ridge 4 ElasticNet	Ridge 3 ElasticNet	Ridge 3	Ridge 3	Ridge 4 ElasticNet
PRIVATE SCORE (TRUNC)	3.22	3.21	3.20	3.24	3.22

Moneyball Analytics

ANY QUESTIONS?



ACKNOWLEDGEMENTS

Portions of the technical design, debugging support, and documentation refinement for this project were assisted by ChatGPT (OpenAI).

All final implementation decisions, data engineering work, analytical modeling, and system integration were performed by the project authors.

Domain Research

Using official, reliable and reputable sources only, research US Major League Baseball statistics, including aggregate statistics such as:

- i) batting average
- ii) on-base percentage
- iii) slugging
- iv) on-base plus slugging, streaks, etc

since the 1900s until present day. The query should provide deep insight into what makes a winning MLB team.

Include possible out-of-the-box variables such as:

- i) ownership changes
- ii) funds injections
- iii) change of home venues or cities
- iv) MLB rule changes, global events (e.g. war)
- v) climate

vi) conditions of state and national governance,
and any other variables that is of value, no matter how remote, that may affect the statistics of the game during those periods.

Hi ChatGPT, for this query, you are an experience MLB statistician and data engineer.

Using the data sources added to this chat ONLY, please provide the variables that contribute to each of the following skills evaluation metrics in MLB.

Exclude the skills evaluation that cannot be derived from the attached data sources.

A) Offensive Evaluation Metrics

1. On-Base Percentage (OBP)
2. Slugging Percentage (SLG)
3. OPS (On-Base Plus Slugging)
4. Launch Angle and Exit Velocity
5. Runs Created
6. Plate Appearances
7. Runs Batted In (RBI)
8. Strikeout rate
9. Pitches per plate appearance

B) Pitching Evaluation Metrics

1. Earned Run Average (ERA)
2. Saves (SV)
3. WHIP (Walks plus Hits per Innings Pitched)
4. Spin Rate and Velocity
5. Hits Allowed and Innings Pitched

C) Defensive and Specialized Metrics

1. Fielding Percentage
2. Range Factor
3. Outfielder Route Efficiency
4. Pitch Framing
5. Catch Probability

D) Comprehensive Value Metrics

1. WAR (Wins Above Replacement)
2. TPR (Total Player Rating)
3. Pythagorean Winning Percentage
4. Adjusted OPS+

Convert them into feature engineering pipelines (Python) to fulfill the below objective:

- Predict the number of games a team will win in a season based on their performance statistics.

Your model should be able to generalize across different eras of baseball.

Evaluation Metric:

Predictions are evaluated using Mean Absolute Error (MAE), which measures the average absolute difference between the predicted wins and actual wins. The lower the MAE, the better the model's performance.

Prediction files should be in the format of the attached 'sample_submission.csv'.

Construct 3 separate ML models below, to run the above prediction in separate sections within one .ipynb notebook file.

1. auto-sklearn (<https://automl.github.io/auto-sklearn/master/>)
2. autoML (run 10 random models one by one and select the best one)
3. autoML ensemble (random select 5 best models to construct to run the predictions)

Please include clear instructions in each model to show users which hyperparameters can be adjusted to achieve desired prediction outcome.

All 3 models should output a prediction csv based on the attached 'sample_submission.csv'.

IMPORTANT: Ensure the models can be executed independently based on environment.yml requirements.

Each output prediction file should be named separately in the format 'prediction_submission_*.csv' where * refers to model 1, 2 or 3 as above.

Finally, create a download link for this .ipynb notebook file available. Do consult me if there are any unclear instructions.

Moneyball Analytics – Appendix

Reverse Engineered Prompt 1 of 2

You are an experienced MLB statistician, data scientist, and ML engineer.

I am working with structured MLB team-season data (train + prediction datasets). Your task is to design a complete, production-ready machine learning pipeline to predict the number of wins (W) for each team-season.

OBJECTIVE

Predict team wins (W) using historical performance statistics.

Evaluation metric:

- Mean Absolute Error (MAE)

The model must generalize across different baseball eras (i.e., avoid temporal leakage).

DATA CONSTRAINTS

- Use ONLY the provided tabular datasets (no external)
- Data includes team-level batting, pitching, and fielding stats
- Assume typical Lahman-style schema (e.g., H, BB, AB, HR, ER, IPouts, R, RA, etc.)
- Prediction dataset has same schema except for target

FEATURE ENGINEERING REQUIREMENTS

Derive only metrics that can be computed from available columns:

Include:

- OBP (simplified), SLG, OPS
- Runs Created (basic)
- PA (approximation)
- Strikeout rate
- ERA (recomputed)
- WHIP
- Pythagorean win %
- Any useful ratio-based features (runs/game, etc.)

Exclude:

- Statcast metrics (launch angle, exit velocity)
- WAR, OPS+, advanced tracking metrics

Also:

- Create era-based features using yearID
- Ensure numerical stability (safe division)

DATA PREPROCESSING

You MUST:

- Convert boolean columns to integers (0/1)
- Handle pandas nullable boolean dtype
- Convert categorical columns using OneHotEncoder
- Impute missing values:
 - numeric → median
 - categorical → most frequent
- Standardize numeric features
- Ensure prediction dataset columns align with training

IMPORTANT:

- Avoid dtype issues that break sklearn (e.g., bool in SimpleImputer)

VALIDATION STRATEGY

Use:

- GroupKFold grouped by yearID

Reason:

- Prevent leakage across seasons
- Ensure generalization across eras

MODEL REQUIREMENTS - Build 3 separate models:

MODEL 1: auto-sklearn

- Use AutoSklearnRegressor
- Make it robust to:
 - temp folder conflicts
 - unsupported parameters (e.g., output_folder in older versions)
- Allow hyperparameters:
 - time_left_for_this_task
 - per_run_time_limit
 - ensemble_size
 - CV folds

MODEL 2: Custom AutoML

- Run N random trials (default ~18)
- Each trial:
 - randomly sample model + hyperparameters
 - evaluate via GroupKFold MAE
- Supported models:
 - Ridge
 - ElasticNet
 - RandomForest
 - ExtraTrees
 - GradientBoosting
 - HistGradientBoosting
 - LightGBM

IMPORTANT:

- Fix max_features bug:
 - numeric values MUST be float (e.g., 0.5, not "0.5")
- Add trial-level exception handling (do not crash loop)

MODEL 3: Ensemble

- Run larger trial pool (~24)
- Select top K models (default 5)
- Prefer diversity across model families
- Combine predictions using:
 - inverse MAE weighting

OUTPUT REQUIREMENTS

Each model must output:

- CSV file matching sample_submission.csv format

File names:

- prediction_submission_1.csv
- prediction_submission_2.csv
- prediction_submission_3.csv

NOTEBOOK REQUIREMENTS

Provide a Jupyter notebook structured into clear cells:

1. Imports
2. Config
3. Data loading
4. Feature engineering
5. Preprocessing utilities
6. Model 2 (AutoML)
7. Model 2 output
8. Model 3 (ensemble)
9. Model 3 output
10. Summary

ROBUSTNESS REQUIREMENTS

Ensure the code:

- handles dtype issues (bool, category, object)
- avoids sklearn parameter errors
- continues execution if a trial fails
- aligns train/test feature columns safely
- is reproducible (random_state control)

HYPERPARAMETER GUIDANCE

Provide recommended defaults for:

- runtime vs accuracy tradeoff
- trial counts
- CV folds
- model-specific parameter ranges

ENVIRONMENT AWARENESS

Assume:

- scikit-learn environment (no bleeding-edge APIs)
- auto-sklearn version may be older (no output_folder support)

DELIVERABLE

1. Complete notebook code (cell-by-cell)
2. No missing dependencies
3. No runtime-breaking bugs
4. Ready to copy-paste and run

Focus on:

- Correctness, Robustness
- Reproducibility, practical usability